Computer Networks
Fall 2024/25

Exercise-2

Submission in pairs by Thursday, **12-12-2024.** Submission is done by uploading your work to the course Moodle website. >>> No late submissions will be accepted
The name of the submitted file must be Exercise2_username1_username2.[suffix].
For example: Exercise2_israel.israeli_david.davidi.zip (with dots). The first and last names of both students must appear. Only one student should submit the assignment (not both). It Covers Chapter 2 from the textbook and Lecture 5 about the consensus problem. Please notice that once the grades for an exercise are published, appeals can only be submitted in the following five days. The grader is Mr. Daniel Ohayon, and you may contact him by email at *reichman.computer.networks@gmail.com.*

Problem 1 (20 points).
In this problem, we consider the performance of HTTP, comparing non-persistent HTTP with persistent HTTP. Suppose the HTML page your browser wants to download is **10K** bits long and contains **10** embedded images, each of **10K** bits in length. The page and the **10** images are all stored on the same Server, which has a **300 msec** RTT from your browser. We will abstract the network path between your browser and the web server as a **100Mbps** link ($100*10^6$ bits per second). You can assume that the time it takes to transmit a GET message into the path (put in on the link) is zero, but you should account for the time it takes to transmit the base HTML file and the embedded objects into the link. In your answer below, make sure to take into account the time needed to set up a TCP connection. Assume that your browser knows the exact IP address of the Server in advance.

1. Assuming non-persistent HTTP with no parallel connections. What is the response time from the point when the user requests the page to the point in time when the page and its embedded objects are displayed?

2. Again, assume non-persistent HTTP, but now assume that after the browser gets the HTML page, it can open as many parallel TCP connections to the Server as it wants. What is the response? Assume each parallel connection gets full bandwidth,

3. Now assume persistent HTTP with no pipelining and no parallel connections. What is the response time?

4. Now assume persistent HTTP with pipelining and with no parallel connections. What is the response time?

Give all the answers in msec, and justify your answers.
Answer:

Question	response time (in msec)
1	
2	
3	
4	

Problem 2 (20 points).

This question aims to get you comfortable retrieving and reading some RFCs.

1. Which protocol is specified in RFC 7231; in what section is **CONNECT** defined?
2. Which protocol is specified in RFC 7540; in what section are **Header Compression and Decompression** defined?
3. What is the title of RFC 6265; in what section is **The Cookie Header** explained?
4. What is the title of RFC 3596; in what section are **security considerations** discussed?
5. Which protocol is specified in RFC 9000? In which section is **Flow Control** discussed?
6. What is the title of RFC 3596; in what section the **AAAA** record type is defined?
7. What is the full name of RFC 6797? In which section is the **Threat Model** discussed?
8. What is the full name of RFC 9114? In which section is **Server Push** explained?
9. Which protocol is specified in RFC 2246; in which section the **handshake protocol** is defined? Who can send a hello message?
10. Which protocol is specified in RFC 1730; in what section the **NOOP** Command is defined?

The length of your answer for each one of the above questions should be one (preferred) or two lines.

Problem 3 (20 points).

Download the Windows version of **ncat** from http://nmap.org/ncat/ (direct link: http://nmap.org/dist/ncat-portable-5.59BETA1.zip). Your Anti-Virus software may identify **ncat** as a virus; in such a case, turn off your anti-virus (**ncat** is not a virus).

1. It is possible to run two servers on the same computer simultaneously, one over UDP and the other over TCP, using the **same** port number. Try it using **ncat** (or **nc** or **netcat**, whatever variant your OS supports) and send us a screenshot. (**ncat -h** will show you all possible options. Use **-l** for running a server and **-u** to run it over UDP.)

2. Get **ncat** listening on port **4444** over **TCP** using the command ``**ncat –v –k –l 4444".** On the same machine, use your browser to access the URL http://127.0.0.1:4444/ from **three** different **Tab**s. Your browser should now send two HTTP GET request messages to the **ncat** Server, and your Server should display the messages. Use **ncat** to return the string "**CN 2024**". Are the HTTP requests sent from the same (source) ports? Which of the Tabs gets the response? Send us a screen capture of the browser's HTTP requests and **ncat's** HTTP response.

Problem 4 (20 points).
Write a simple server program (in Java) that runs over **TCP** using port **8888** that accepts lines of input from a client, and first prints "Welcome to CN 2024: Exercise 2: <YOUR STUDENT names>" and then all the accepted lines onto the Server's standard output.

1. Set the proxy server information in your browser to the host running your server program and configure the port number appropriately. Using your browser, try accessing the URL: http://www.google.com. Your browser should now send either a GET request message OR a CONNECT request to your Server, and your Server should display the message. (To understand why your Server got a CONNECT message, you may read or see the following video about HTTP Strict Transport Security -- https://www.youtube.com/watch?v=-MWqSD2_37E
You can also disable HTST in your browser and see what happens).

2. On the same machine, run a **ncat** client to connect to **your** Server program listening on port 8888 over **TCP.** Use the **ncat** client to send a message of **precisely 20** lines to your Server. Your Server should display the messages.

Please send us screen captures and the program code (only .java files, without .class files) in a zip file without directories.

Problem 5 -- Lecture #5 (20 points).

1. Consider the consensus algorithm for crash failures. Assume that (1) in each round, either no process fails or at least **5** processes fail, (2) t =40, and (3) n= 70. Assume that the condition "**until** *round* = t+1" is replaced with "**until** *round* = k + 1". What is the smallest k for which the algorithm is still correct?

2. Consider the algorithm for binary consensus in the case of byzantine failures. Suppose $n = 26$ and $t = 7$. (Note that in this case, it does not hold that $n \geq 4t + 1$). Assume that all the correct processes start with the same input value 0. Is it possible that all the correct processes will reach an agreement at the end and that the agreement's value will be 1? (Recall that $s = n\text{-}t$ = 19.)

3. Consider the algorithm for binary consensus in the case of byzantine failures. Suppose $n = 45$ and $t = 11$, and we change the number of iterations from $t + 1$ to $t + 11$.
Assume that all the correct processes, except one, start with the same input value 0. Is it possible that all the correct processes will reach an agreement at the end and that the agreement's value will be 1? (Recall that $s = n\text{-}t$ = 34.)

Briefly justify your answers.